NonOpt Manual
Version 2.2

Frank E. Curtis

July 31, 2026

Contents

1 Introduction **1**
- 1.1 Algorithm Overview . . . 2
- 1.2 Availability . . . 2

2 Installation **2**
- 2.1 Obtaining NonOpt . . . 2
- 2.2 Compiling NonOpt . . . 2
- 2.3 Compiling NonOpt with CMake . . . 3
- 2.4 Testing the Installation . . . 3
- 2.5 Quick Start . . . 3

3 Interfaces **4**
- 3.1 C++ Interface . . . 4
  - 3.1.1 Implementing a Problem . . . 4
  - 3.1.2 Solving a Problem . . . 4
- 3.2 Python Interface . . . 4
  - 3.2.1 Building the Python Module . . . 4
  - 3.2.2 Implementing a Problem in Python . . . 5
  - 3.2.3 Solving a Problem in Python . . . 5
- 3.3 AMPL Interface . . . 7
- 3.4 Changing Options . . . 7

4 NonOpt Reporter Output **9**

1 Introduction

NonOpt (Nonconvex, Nonsmooth Optimizer) is an open-source C++ software package for solving unconstrained minimization problems. It is designed to locate a minimizer, or at least a (nearly) stationary point, of an objective function $f : \mathbb{R}^n \to \mathbb{R}$, which is assumed to be locally Lipschitz. That is, given such an objective function f, the software contains algorithms that are designed to solve problems of the form

$$\min_{x \in \mathbb{R}^n} \; f(x).$$

The objective function f can be nonlinear, nonconvex, and/or nonsmooth. For further information, see [5].

This manual documents NonOpt version 2.2.

1.1 Algorithm Overview

NonOpt contains implementations of a collection of algorithms, including gradient-sampling and proximal-bundle methods, each equipped with quasi-Newton techniques for improving practical performance. You can choose between different strategies for the search direction computation, approximate Hessian approximation update, line search, point-set update, symmetric matrix storage, and termination. For more information on the implemented algorithms, see [5], which in turn references the algorithms developed in [1, 2, 3, 4].

1.2 Availability

NonOpt is written in C++ and is available on GitHub here—see also the NonOpt homepage here—under the MIT open source license. This puts very few restrictions on the use and reuse of the software. This license has been selected since the goal of open-source software is continuous development and improvement. It is only asked that if you use the software in work that leads to a publication, then you cite the article [5], and potentially (a subset of) the articles [1, 2, 3, 4], when applicable. Writing effective and computationally efficient software requires a lot of time and effort, so citations are appreciated!

NonOpt can be used from C++ directly, from Python via the `nonoptpy` module, or from AMPL via a solver executable. These three interfaces are described in §3.

2 Installation

NonOpt has been written with the intention of having it be Unix-based platform and compiler independent, but there are no guarantees. Comments and/or questions can be sent to Frank E. Curtis, although there are no guarantees that all issues will be resolvable. (Please do not e-mail with any inquiries pertaining to installing NonOpt on Microsoft Windows! You are on your own for that. Sorry!)

2.1 Obtaining NonOpt

NonOpt is available on GitHub here; see also the NonOpt homepage here.

2.2 Compiling NonOpt

A C++ compiler is required to compile the NonOpt library. Building NonOpt also requires:

- BLAS (Basic Linear Algebra Subroutines). Many operating systems provide precompiled and optimized libraries for these subroutines. One could also obtain the source code from www.netlib.org. However, it is recommended to use an optimized BLAS implementation.
- LAPACK (Linear Algebra PACKage). As with BLAS, many operating systems provide precompiled and optimized libraries, but the source code can also be obtained from www.netlib.org.

After obtaining NonOpt, an environment variable needs to be set:

- `$NONOPTDIR` should point to the NonOpt root directory of the distribution that you obtained. This root directory is the one containing the `README.md` file for the distribution.

After this environment variable has been set, the NonOpt library can be built by issuing the commands below. Here, and throughout this document, "`>`" indicates your terminal prompt.

```
> cd $NONOPTDIR/NonOpt
> make
```

This should make the libraries in each of the `src`, `problems`, and `tests` subdirectories. It should also make two executables, `runExperiment` and `solveProblem`, in the `exes` subdirectory, as well as a few executables in the `mpc` subdirectory, which are provided to reproduce the numerical results for the experiments in [5].

The Makefiles build with optimizations enabled (`-O3 -DNDEBUG`) by default. An unoptimized build with debugging symbols can be obtained in any of these directories by issuing `make debug` instead of `make`.

The Python and AMPL interfaces are built separately; see §3.2 and §3.3, respectively.

2.3 Compiling NonOpt with CMake

As an alternative to the Makefiles described in §2.2, NonOpt can be built with CMake (version 3.15 or later). The two build systems coexist; neither interferes with the other. From the NonOpt root directory, issue:

```
> cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
> cmake --build build -j
```

CMake locates BLAS and LAPACK automatically. To request a specific implementation, configure with, e.g., `-DBLA_VENDOR=OpenBLAS`. Libraries are placed in `build/lib` and executables in `build/bin`. The build defaults to `Release` if no build type is specified. NonOpt can also be installed, and then located by other CMake projects via `find_package(NonOpt)`, by issuing:

```
> cmake --install build --prefix /your/install/prefix
```

2.4 Testing the Installation

If the compilation has been successful, then a few unit tests and a minimization test can be run by issuing:

```
> cd $NONOPTDIR/NonOpt/tests
> ./testAll
```

Standard output will reveal the results of the unit tests. If all unit tests are successful, then the script will continue to run a minimization test for a test problem that is implemented in `$NONOPTDIR/NonOpt/problems`. If a unit test fails, then the minimization test is not run. In this case, it is possible (although not necessarily the case) that NonOpt will not run properly, or at least the performance that you will obtain by using NonOpt will not be as good as expected. More detailed output from each unit test can be obtained by running each unit test separately. For example, the symmetric matrix class can be tested by issuing:

```
> cd $NONOPTDIR/NonOpt/tests
> ./testSymmetricMatrix
```

The last line of the detailed output indicates whether or not the test was successful. Each test executable, including `testAll`, also returns a process exit code of zero on success and nonzero on failure, so the tests can be run from a script.

If NonOpt was built with CMake as in §2.3, then the same tests are registered with CTest and can all be run by issuing:

```
> ctest --test-dir build --output-on-failure
```

2.5 Quick Start

For the impatient, the following builds NonOpt and solves one of the provided test problems:

```
> export NONOPTDIR=/path/to/NonOpt
> cd $NONOPTDIR/NonOpt
> make
> cd $NONOPTDIR/NonOpt/exes
> ./solveProblem MaxQ
```

The output that this produces is described in §4. To solve your own problem instead, see §3.

3 Interfaces

NonOpt can be used from C++ (§3.1), from Python (§3.2), or from AMPL (§3.3). In all cases, the algorithmic behavior is controlled by the same set of options, which are described in §3.4.

3.1 C++ Interface

NonOpt comes with a set of test problems in the `$NONOPTDIR/NonOpt/problems` directory. The easiest way to interface your problem to NonOpt is to follow one of these as an example.

3.1.1 Implementing a Problem

For example, consider `MaxQ` implemented in `MaxQ.hpp` and `MaxQ.cpp` in `$NONOPTDIR/NonOpt/problems`. This problem is scalable, with the input `n` to the constructor dictating the dimension of the problem to be solved. If your problem is not scalable, then the value of `number_of_variables` can be hard-coded. The remainder of an implementation of a problem can be understood in this example. Overall, the functions that need to be implemented are those dictated in the header file `$NONOPTDIR/NonOpt/src/NonOptProblem.hpp`.

`Note:` It is extremely easy to set up NonOpt for failure if you have bugs in your objective and/or derivative implementations. NonOpt comes equipped with a derivative checker that you can use to test your implementation of your `evaluateGradient` function with respect to your `evaluateObjective` function. See §3.4 for instructions on turning on the derivative checker when your problem is being solved.

3.1.2 Solving a Problem

Suppose your problem is implemented in `YourProblem.hpp` and `YourProblem.cpp`. If these files are located in `$NONOPTDIR/NonOpt/problems`, then running `make` in this directory will add `YourProblem` to the `libNonOptProblems.a` library that is built in this directory. Your problem can then be solved by adding it to the list of problems in `solveProblem.cpp` in `$NONOPTDIR/NonOpt/exes`, then issuing:

```
> cd $NONOPTDIR/NonOpt/exes
> ./solveProblem YourProblem
```

You might also implement your own executable to set up and solve your problem.

3.2 Python Interface

NonOpt provides a Python interface, built with pybind11, that exposes the solver as a module named `nonoptpy`. A problem is defined in Python by subclassing `nonoptpy.Problem`, so no C++ needs to be written. Objective and gradient values are exchanged as NumPy arrays.

3.2.1 Building the Python Module

In addition to `$NONOPTDIR`, building the Python module requires that pybind11 and NumPy be installed, and that an environment variable point to the pybind11 headers:

- `$PYBINDDIR` should point to the pybind11 include directory, as reported by `python3 -m pybind11 --includes`.

The module can then be built by issuing:

```
> pip install pybind11 numpy
> cd $NONOPTDIR/NonOpt/python
> make
```

This produces a shared object named `nonoptpy` followed by the extension suffix for your Python installation, e.g., `nonoptpy.cpython-311-darwin.so`. The module is importable from any directory on your Python path; the simplest approach is to run your scripts from `$NONOPTDIR/NonOpt/python`.

3.2.2 Implementing a Problem in Python

A problem is implemented by subclassing `nonoptpy.Problem` and defining the methods in Table 1. Note that the Python signatures are not identical to their C++ counterparts: values that are returned through output arguments in C++ are, in several cases, returned directly in Python or written in place into a NumPy array.

Method	Description
`numberOfVariables(self)`	Returns the number of variables n directly, as an integer.
`initialPoint(self, n, x)`	Writes the initial point in place into the NumPy array `x`. Returns `True` on success.
`evaluateObjective(self, x)`	Returns $f(x)$ directly, as a float. Note that this method takes only `x`; it does *not* take `n`.
`evaluateGradient(self, n, x, g)`	Writes the gradient in place into the NumPy array `g`. Returns `True` on success.
`evaluateObjectiveAndGradient(self, n, x, f, g)`	Writes the objective value into `f[0]`, where `f` is a *list*, and the gradient in place into `g`. Returns `True` on success. Used only when the option `evaluate_function_with_gradient` is `true`.
`finalizeSolution(self, n, x, f, g)`	Called once with the final point, objective value, and gradient. Returns `True` on success. This method is required: it must be defined even if it does nothing but `return True`.

Table 1: Methods of a `nonoptpy.Problem` subclass.

Arrays that are written in place (`x` in `initialPoint`, `g` in the gradient methods) are views onto NonOpt's own memory. They should be filled element-wise or with in-place NumPy operations such as `g.fill(0)`; rebinding the name, as in `g = np.zeros(n)`, will *not* communicate any values back to the solver.

3.2.3 Solving a Problem in Python

The following complete example implements and solves the `MaxQ` problem. It is available, along with a Rosenbrock example, in `$NONOPTDIR/NonOpt/python`.

```
import nonoptpy as nonopt
import numpy as np

class MaxQ(nonopt.Problem):

  def numberOfVariables(self):
    return 1000

  def initialPoint(self, n, x):
    for i in range(n//2):
      x[i] = i + 1
    for i in range(n//2,n):
      x[i] = -i - 1
    return True

  def evaluateObjective(self, x):
    return np.max(np.abs(x))**2
```

```
  def evaluateGradient(self, n, x, g):
    index = np.argmax(np.abs(x))
    g.fill(0)
    g[index] = 2 * x[index]
    return True

  def evaluateObjectiveAndGradient(self, n, x, f, g):
    f[0] = np.max(np.abs(x))**2
    index = np.argmax(np.abs(x))
    g.fill(0)
    g[index] = 2 * x[index]
    return True

  def finalizeSolution(self, n, x, f, g):
    return True

# Declare solver
solver = nonopt.Solver()

# Modify options from file
solver.options().modifyOptionsFromFile("nonoptpy.opt")

# Add file report
solver.reporter().addFileReport("report", "nonoptpy.out",
                                nonopt.ReportType.R_NL,
                                nonopt.ReportLevel.R_PER_ITERATION)

# Optimize
solver.optimize(MaxQ())

# Print a few statistics
print(f"Final objective ..... : {solver.objective()}")
print(f"Iterations .......... : {solver.iterations()}")
print(f"CPU seconds ......... : {solver.time()}")
print(f"Solver status ....... : {solver.status()}")

# Retrieve the solution
x = np.zeros(solver.numberOfVariables())
solver.solution(x)
```

The `nonoptpy.Solver` object exposes the following methods. The statistics correspond to the quantities reported at the end of a run; see §4.

- `optimize(problem)`: solves the given `Problem` instance.
- `solution(x)`: writes the final iterate into the preallocated NumPy array `x`, which must have length `numberOfVariables()`.
- `options()`: returns the `Options` object; see §3.4.
- `reporter()`: returns the `Reporter` object, whose `addFileReport(name, file, type, level)` method directs output to a file. The arguments `type` and `level` are values of the enumerations `nonoptpy.ReportType` (`R_NL` or `R_QP`) and `nonoptpy.ReportLevel` (`R_NONE`, `R_BASIC`, `R_PER_ITERATION`, or `R_PER_INNER_ITERATION`).

- `objective()`, `stationarityRadius()`, `numberOfVariables()`: solution quantities.
- `iterations()`, `totalInnerIterations()`, `totalQPIterations()`, `functionEvaluations()`, `gradientEvaluations()` iteration and evaluation counts.
- `time()`, `timeEvaluations()`, `timeNonOpt()`: CPU seconds in total, in function and gradient evaluations, and in NonOpt itself.
- `status()`: a value of the enumeration `nonoptpy.NonOpt_Status`, e.g., `NONOPT_SUCCESS`. These correspond to the EXIT messages listed in §4.

3.3 AMPL Interface

NonOpt can be used as a solver for problems modeled in AMPL through the AMPL Solver Library (ASL). In addition to `$NONOPTDIR`, two further environment variables are required:

- `$AMPLDIR` should point to the directory containing the compiled ASL, i.e., the directory containing `amplsolver.a` and the ASL headers.
- `$LAPACKDIR` should point to the directory containing your BLAS and LAPACK libraries.

The interface is then built by issuing:

```
> cd $NONOPTDIR/NonOpt/ampl
> make
```

This produces the library `libAMPLProblem.a` and the executable `solveAMPLProblem`. Given a problem written in AMPL and exported to a `.nl` file, say `YourProblem.nl`, the problem is solved by issuing:

```
> cd $NONOPTDIR/NonOpt/ampl
> ./solveAMPLProblem YourProblem
```

Note that the `.nl` suffix is appended automatically and should be omitted from the argument.

3.4 Changing Options

NonOpt is designed to be flexible, offering various options, each of which can be changed from its default value, if desired. For a complete list of options, along with their types, bounds, and default values, please see the `$NONOPTDIR/NonOpt-Manual/options.txt` file. A selection of the most commonly used options is given in Table 2. Some safeguards have been added so that you cannot set an option to an illegal value, although it is possible that you could set an option to a value that will cause the optimizer to behave in strange ways. If you provide a legal value (and the `Reporter` is issuing output to a report at a sufficient level), then a note will be printed (to a report) that the value for an option has been changed. If you attempt to set an option to an illegal value, then similarly a note will be printed saying that a change was attempted, but the attempt was not successful (due to the provided value being illegal). Options should be set with caution, or at least with sufficient knowledge of the underlying algorithms.

The value for an option can be changed in one of two ways:

- In an executable. After a `NonOptSolver` object has been declared (via `NonOptSolver nonopt;`), one can modify the value of an option via a statement such as:

```
nonopt.options()->modifyDoubleValue("cpu_time_limit", 600.0);
```

Option	Type	Default
print_level	integer	1
print_level_file	integer	0
print_file_name	string	nonopt.out
cpu_time_limit	double	1e+04
iteration_limit	integer	1e+06
function_evaluation_limit	integer	Infinity
gradient_evaluation_limit	integer	Infinity
stationarity_tolerance	double	1e-04
direction_computation	string	CuttingPlane
approximate_hessian_update	string	BFGS
line_search	string	WeakWolfe
point_set_update	string	Proximity
qp_solver_small_scale	string	DualActiveSet
qp_solver_large_scale	string	InteriorPoint
symmetric_matrix	string	Dense
termination	string	Basic
derivative_checker	string	FiniteDifference
DEFD_check_derivatives	bool	false

Table 2: Commonly used options. See `options.txt` for the complete list.

The functions `modifyBoolValue`, `modifyDoubleValue`, `modifyIntegerValue`, and `modifyStringValue` all have the same form, with the two arguments indicating the name of the option and the value to which you would like to set it, respectively.

- In an options file. After a `NonOptSolver` object has been declared, one can modify the value of *multiple* options via a statement such as:

```
nonopt.options()->modifyOptionsFromFile("nonopt.opt");
```

Here, `nonopt.opt` represents the name of a file containing a list of options and the values to which you would like to set them. For example, the contents of the file might be:

```
cpu_time_limit 600.0
iteration_limit 1e+4
```

Each line should contain only the name of an option followed by the desired value.

The same methods are available from Python on the object returned by `solver.options()`, with identical names and arguments, e.g.:

```
solver.options().modifyDoubleValue("cpu_time_limit", 600.0)
solver.options().modifyOptionsFromFile("nonoptpy.opt")
```

As previously mentioned, a useful option is to turn on the derivative checker, which is off by default. The default derivative checker option is `FiniteDifference`. Assuming this is the derivative checker that has been chosen, the derivative checker can be turned on by the statement:

```
nonopt.options()->modifyBoolValue("DEFD_check_derivatives", true);
```

or within an options file by including the line:

`DEFD_check_derivatives true`

The derivative checker can be computationally expensive, so it is not recommended to run the derivative checker when performing timed experiments. But it is the best first step for debugging!

4 NonOpt Reporter Output

The option `print_level` determines the level of output to be printed to standard output. Setting this level to `0` means that no information will be printed. Setting this level to `1` means that a basic amount of information will be printed, such as in the example below. The information at this level is self-explanatory, perhaps except for the EXIT messages, which are summarized after the example.

```
+-------------------------------------------------------------+
|       NonOpt = Nonlinear/Nonconvex/Nonsmooth Optimizer      |
| NonOpt is released as open source code under the MIT License |
|     Please visit: https://github.com/frankecurtis/NonOpt    |
+-------------------------------------------------------------+

This is NonOpt version 2.2

Number of variables.................. : 1000
Initial objective.................... : 6.908755e+00
Initial objective (unscaled)......... : 6.908755e+00

Approximate Hessian update strategy.. : BFGS
Derivative checker strategy.......... : FiniteDifference
Direction computation strategy....... : CuttingPlane
Line search strategy................. : WeakWolfe
Point set update strategy............ : Proximity
QP solver (small scale) strategy..... : DualActiveSet
QP solver (large scale) strategy..... : InteriorPoint
Symmetric matrix strategy............ : Dense
Termination strategy................. : Basic

EXIT: Stationary point found.

Objective............................ : 3.008077e-09
Objective (unscaled)................. : 3.008077e-09

Number of iterations................. : 22
Number of inner iterations........... : 25
Number of QP iterations.............. : 29
Number of function evaluations....... : 414
Number of gradient evaluations....... : 347

CPU seconds.......................... : 0.108261
CPU seconds in evaluations........... : 0.006546
CPU seconds in direction computations : 0.065522
CPU seconds in line searches......... : 0.008766
```

There are various EXIT messages that may be printed, which indicate the reason that the `NonOptSolver` terminated. Some messages end with the phrase "`This wasn't supposed to happen!`" If any of those are

printed, then we ask the user to contact Frank E. Curtis, since that was not supposed to happen! With respect to the EXIT messages that are intended to be printed, their meanings are the following.

- `Stationary point found`: A point has been found satisfying the stationarity conditions; see, e.g., [4].
- `Objective not improving sufficiently`: The objective has not improved sufficiently in recent iterations, indicating that the algorithm is not likely to make substantial further progress.
- `Objective tolerance reached`: A point has been found whose objective value is below a tolerance that has been provided by the user.
- `CPU time limit reached`: The CPU time limit has been reached.
- `Iterates seem to be diverging`: A point has been found whose norm is above a tolerance.
- `Iteration limit reached`: The iteration limit has been reached.
- `Function evaluation limit reached`: The function evaluation limit has been reached.
- `Gradient evaluation limit reached`: The gradient evaluation limit has been reached.
- `Approximate Hessian update failure`: A failure occurred, due to a numerical tolerance being violated, in the update of the approximate Hessian matrix. By default, the code would not terminate based on such a tolerance, so this would only occur if a user turns this feature on as a termination condition.
- `Derivative checker failure`: A failure occurred, due to a numerical tolerance being violated, in the derivative checker. By default, the code would not terminate based on such a tolerance, so this would only occur if a user turns this feature on as a termination condition.
- `Direction computation failure`: A failure occurred, due to a numerical tolerance being violated, in the direction computation. By default, the code would not terminate based on such a tolerance, so this would only occur if a user turns this feature on as a termination condition.
- `Function evaluation failure! Check definition of problem`: An error occurred in the function evaluation code provided by the user.
- `Gradient evaluation failure! Check definition of problem`: An error occurred in the gradient evaluation code provided by the user.
- `Line search failure`: A failure occurred, due to a numerical tolerance being violated, in the line search. By default, the code would not terminate based on such a tolerance, so this would only occur if a user turns this feature on as a termination condition.
- `Point set update failure`: A failure occurred, due to a numerical tolerance being violated, in the point set update. By default, the code would not terminate based on such a tolerance, so this would only occur if a user turns this feature on as a termination condition.
- `Problem data read failure! Check definition of problem`: An error occurred in the code provided by the user.
- `Termination check failure`: A failure occurred, due to a numerical tolerance being violated, in the termination checker. By default, the code would not terminate based on such a tolerance, so this would only occur if a user turns this feature on as a termination condition.

When NonOpt is used from Python, the same information is available programmatically through `solver.status()`, which returns a value of the `nonoptpy.NonOpt_Status` enumeration corresponding to the EXIT message; see §3.2.

Setting `print_level` to `2` means that per-iteration output will be printed in addition to the basic output shown above. A header line is printed periodically to indicate the meaning of the values in each column of the per-iteration output. Further information about these quantities is provided in the list below. The per-iteration output when the `direction_computation` option is set to either `CuttingPlane` or `GradientCombination` is exactly the same, whereas when this option is set to `Gradient` the column `L?` is omitted since the small-scale QP subproblem solver is always employed for this strategy.

- `Iter.`: iteration index
- `Objective`: objective function value
- `St. Rad.`: stationarity radius
- `Tr. Rad.`: trust region radius
- `|Pts|`: cardinality of point set
- `In Its`: inner iteration index in direction computation method
- `QP Pts`: cardinality of points used to build QP subproblem
- `L?`: indicator that QP is large scale (`1`) or not (`0`)
- `QP Its`: iterations required to solve QP subproblem
- `S?`: termination status of QP subproblem solver (`0` indicates success)
- `QP KKT`: QP subproblem solution KKT error
- `|Step|`: norm of search direction
- `|Step|_H`: norm of search direction, with norm defined by approximate Hessian
- `|Grad.|`: norm of objective gradient
- `|G. Cmb.|`: norm of combination of gradients corresponding to QP subproblem solution
- `Stepsize`: step size from line search
- `Up. Fact.`: approximate Hessian update factor
- `U?`: indicator that approximate Hessian update is being updated (`1`) or not (`0`)

References

[1] Frank E. Curtis and Minhan Li. Gradient Sampling Methods with Inexact Subproblem Solutions and Gradient Aggregation. *INFORMS Journal on Optimization*, 4(4):347–445, 2022.

[2] Frank E. Curtis and Xiaocun Que. An Adaptive Gradient Sampling Algorithm for Nonsmooth Optimization. *Optimization Methods and Software*, 28(6):1302–1324, 2013.

[3] Frank E. Curtis and Xiaocun Que. A Quasi-Newton Algorithm for Nonconvex, Nonsmooth Optimization with Global Convergence Guarantees. *Mathematical Programming Computation*, 7(4):399–428, 2015.

[4] Frank E. Curtis, Daniel P. Robinson, and Baoyu Zhou. A Self-Correcting Variable-Metric Algorithm Framework for Nonsmooth Optimization. *IMA Journal of Numerical Analysis*, 40(2):1154–1187, 2020.

[5] Frank E. Curtis and Lara Zebiane. NonOpt: Nonconvex, Nonsmooth Optimizer. https://arxiv.org/abs/2503.22826, 2025. arXiv 2503.22826.